FEATURE ADDENDUM

Admin Dashboard

VIGILANTE Resume Tailor

Safe, honest, read-only monitoring of users and usage — admin access only

Type	Fits Into	Access	Data Policy
Feature Addendum	**Phase 4 (UI) + Phase 5 (Security)**	**Single Designated Admin Only**	**Read-Only. No PII export.**

A. How This Addendum Integrates with the Build Prompt

This document is a self-contained addendum to the VIGILANTE Resume Tailor Full Build Prompt. The agent must incorporate every specification in this document into the appropriate phases of the existing plan. Nothing in the main build prompt is overridden — this addendum only adds new requirements.

Integration Instructions for the Agent

- Section B (Database) — Apply as Migration 004 immediately after Migration 003 in Phase 0.
- Section C (Environment) — Add the new env vars to .env.local and .env.example immediately in Phase 0.
- Section D (Security Model) — Implement the admin role check middleware in Phase 1 alongside existing auth middleware.
- Section E (API Routes) — Build admin API routes in Phase 2 alongside the existing API routes.
- Section F (UI) — Build the Admin Dashboard pages in Phase 4 alongside the user-facing pages.
- Section G (Tests) — Write all admin tests in the appropriate test files during the corresponding phase.
- Section H (Privacy Principles) — These are non-negotiable constraints that apply to every admin feature.
- Section I (Phase Report) — Add admin-specific completion criteria to each affected phase report.

B. Privacy & Ethics Principles — The Foundation

Before any technical specification: the Admin Dashboard is built on a foundation of honest, minimal, and responsible data access. These principles govern every decision in this feature. They are not preferences — they are constraints.

[MUST] — Privacy Principles — All Non-Negotiable

1. READ-ONLY: The admin dashboard NEVER modifies, deletes, or exports user data. It is purely observational.
2. NO RAW RESUME CONTENT: Admin cannot view the actual text of any user's resume or job description. Only metadata (lengths, timestamps, scores) is visible.
3. NO PII DOWNLOAD: The admin interface has no export button, no CSV download, no bulk data extraction of any kind.
4. MINIMAL PII DISPLAY: Admin sees user email addresses (needed for account management) but never full resume content, job descriptions, or any sensitive personal career information.
5. SINGLE ADMIN: There is exactly one admin account, identified by a hardcoded email address stored in an environment variable. No admin role can be granted via the UI or API.
6. AUDIT TRAIL: Every admin page view and every admin API call is logged server-side with timestamp and the admin's user_id. These logs are written to a dedicated admin_audit_log table.
7. NO IMPERSONATION: Admin cannot log in as, act as, or view data as if they were another user.
8. TRANSPARENT INTENT: The admin dashboard is for operational monitoring only — usage patterns, error rates, system health. Not for reading users' career data.
9. DATA MINIMIZATION: Aggregated/anonymized metrics are preferred over per-user data wherever possible.
10. DISPLAY NOTICE: A persistent notice on every admin page: 'Admin view — all access is logged.'

C. Database Addendum

C.1 Migration 004 — Admin Role & Audit Log

```
-- ── Admin role column on profiles ──────────────────────────────
-- IMPORTANT: This column is set manually in the Supabase dashboard
-- or via a one-time SQL command. It is NEVER settable via any API.
ALTER TABLE public.profiles
  ADD COLUMN is_admin BOOLEAN NOT NULL DEFAULT FALSE;

-- One-time command to grant admin to the designated user:
-- Run this manually in Supabase SQL editor AFTER creating your account.
-- UPDATE public.profiles SET is_admin = TRUE WHERE email = 'your-
admin@email.com';

-- ── Admin audit log ────────────────────────────────────────────
```

```
CREATE TABLE public.admin_audit_log (
  id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  admin_id     UUID NOT NULL REFERENCES public.profiles(id),
  action       TEXT NOT NULL,   -- e.g. 'VIEW_USERS_LIST', 'VIEW_USER_DETAIL'
  target_id    UUID,            -- user_id being viewed, if applicable
  metadata     JSONB,           -- additional context (page, filters used,
etc.)
  ip_address   TEXT,            -- from request headers
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_audit_admin_id ON public.admin_audit_log(admin_id);
CREATE INDEX idx_audit_created_at ON public.admin_audit_log(created_at DESC);

-- ── RLS on audit log ──────────────────────────────────────────
ALTER TABLE public.admin_audit_log ENABLE ROW LEVEL SECURITY;

-- Only service role (server-side API) can insert
-- Admin can read their own audit entries via API (not direct DB)
-- No user (including admin) can delete audit logs
CREATE POLICY "audit_no_direct_access" ON public.admin_audit_log
  FOR ALL USING (FALSE);  -- Block all direct client access; API layer only

-- ── Usage stats view (anonymized) ──────────────────────────────
-- A DB view for aggregated stats — no per-user PII
CREATE OR REPLACE VIEW public.usage_stats AS
SELECT
  DATE_TRUNC('day', r.created_at)  AS day,
  COUNT(DISTINCT r.user_id)        AS active_users,
  COUNT(r.id)                      AS total_tailorings,
  AVG(r.ats_score)                 AS avg_ats_score,
  COUNT(CASE WHEN r.email_sent THEN 1 END) AS emails_sent,
  COUNT(CASE WHEN r.language = 'ar' THEN 1 END) AS arabic_tailorings,
  COUNT(CASE WHEN r.language = 'en' THEN 1 END) AS english_tailorings
FROM public.resumes r
GROUP BY DATE_TRUNC('day', r.created_at)
ORDER BY day DESC;

-- Grant service role access to the view
GRANT SELECT ON public.usage_stats TO service_role;
```

ACTION REQUIRED — Admin Account Setup

After applying Migration 004 and creating your account in the app:

STEP 1: Sign up for your account in the app using the email address you want as admin.
STEP 2: Go to Supabase Dashboard → SQL Editor and run:
UPDATE public.profiles SET is_admin = TRUE WHERE email = 'your-email@domain.com';
STEP 3: Add your admin email to .env.local:
ADMIN_EMAIL=your-email@domain.com
STEP 4: Confirm with the agent when both steps are done.

⚠ The is_admin flag can ONLY be set via direct SQL. No API endpoint can grant or revoke it.

⚠ There must only ever be ONE row with is_admin = TRUE.

D. Environment Variables — New Additions

Add these to .env.local and .env.example immediately in Phase 0 alongside the existing variables.

```
# ── Admin Configuration ──────────────────────────────────────
# The single designated admin email address.
# Must match exactly the email used to create the admin Supabase account.
# NEVER expose this via NEXT_PUBLIC_* prefix.
ADMIN_EMAIL=

# Admin session secret for additional HMAC verification of admin API calls.
# Generate with: openssl rand -hex 32
ADMIN_HMAC_SECRET=
```

E. Admin Security Model

E.1 Three-Layer Access Control

Admin access is enforced at three independent layers. All three must pass for any admin request to succeed. Failure at any layer returns 403 immediately.

Layer	Check	Implementation
1	Valid Supabase session	Standard Supabase auth check via server client. User must be authenticated. If not → 401.
2	is_admin = TRUE in DB	Server-side query: SELECT is_admin FROM profiles WHERE id = auth.uid(). If false or null → 403. This check uses the service role key and cannot be spoofed by the client.
3	Email matches ADMIN_EMAIL env var	Server-side: profile.email === process.env.ADMIN_EMAIL. Provides defense-in-depth — even if is_admin is somehow set on another account, it won't match. If mismatch → 403.

E.2 requireAdmin() — Shared Server Utility

```
// src/lib/admin/require-admin.ts
// Import this at the top of EVERY admin API route. Never skip it.

import { createServerClient } from '@/lib/supabase/server';
import { NextRequest, NextResponse } from 'next/server';
import { logAdminAction } from './audit-logger';

export async function requireAdmin(req: NextRequest): Promise<
  | { adminId: string; adminEmail: string; error: null }
  | { adminId: null; adminEmail: null; error: NextResponse }
> {
  const supabase = createServerClient();

  // Layer 1: Valid session
  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return { adminId: null, adminEmail: null, error: NextResponse.json({ error:
'UNAUTHORIZED' }, { status: 401 }) };
  }

  // Layer 2: is_admin flag (use service role to bypass RLS)
  const { data: profile } = await supabase
    .from('profiles')
    .select('is_admin, email')
    .eq('id', user.id)
    .single();

  if (!profile?.is_admin) {
    return { adminId: null, adminEmail: null, error: NextResponse.json({ error:
'FORBIDDEN' }, { status: 403 }) };
  }

  // Layer 3: Email matches env var
  if (profile.email !== process.env.ADMIN_EMAIL) {
    return { adminId: null, adminEmail: null, error: NextResponse.json({ error:
'FORBIDDEN' }, { status: 403 }) };
  }

  return { adminId: user.id, adminEmail: profile.email, error: null };
}
```

E.3 Audit Logger

```
// src/lib/admin/audit-logger.ts
// Called after EVERY admin API access. Non-optional.

import { createServiceClient } from '@/lib/supabase/service';
import { NextRequest } from 'next/server';

export type AdminAction =
  | 'VIEW_OVERVIEW'
  | 'VIEW_USERS_LIST'
```

```
  | 'VIEW_USER_DETAIL'
  | 'VIEW_USAGE_STATS'
  | 'VIEW_RECENT_ACTIVITY'
  | 'VIEW_ERROR_RATES';

export async function logAdminAction(
  adminId: string,
  action: AdminAction,
  req: NextRequest,
  opts?: { targetId?: string; metadata?: Record<string, unknown> }
): Promise<void> {
  const supabase = createServiceClient(); // service role — bypasses RLS
  const ip = req.headers.get('x-forwarded-for') ?? req.headers.get('x-real-ip')
?? 'unknown';

  await supabase.from('admin_audit_log').insert({
    admin_id:   adminId,
    action,
    target_id:  opts?.targetId ?? null,
    metadata:   opts?.metadata ?? null,
    ip_address: ip,
  });
  // Intentionally fire-and-forget — do not await or block the response.
  // Errors are swallowed here but monitored via Sentry.
}
```

E.4 Admin Middleware — Route Protection

```
// In src/middleware.ts — ADD to existing middleware logic:

// Admin route protection — redirect non-admins from /admin/*
if (request.nextUrl.pathname.startsWith('/admin')) {
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) {
    return NextResponse.redirect(new URL('/auth/login', request.url));
  }
  // NOTE: Full admin verification (is_admin + email check) happens in each API
route.
  // Middleware only checks for a valid session to avoid DB calls on every
request.
  // The API layer is the authoritative security gate.
}
```

F. Admin API Routes — Complete Specifications

All admin API routes live under /api/admin/. Every route calls requireAdmin() as its first action and logAdminAction() before returning any response. No exceptions.

Data Exposure Rules — Apply to ALL Admin Routes

The following fields are NEVER returned by any admin API route, under any circumstances:

- original_text (resume content)
- job_description (job posting text)
- tailored_json (AI-generated resume)
- tailored_pdf_url (resume file URL)
- original_file_url (uploaded file URL)
- keywords_matched / keywords_missing

The following fields ARE permitted in admin API responses:

- id, user_id, created_at, updated_at
- ats_score, language, template_used
- job_title, company_name (metadata)
- email_sent, tailoring_strength
- original_filename (filename only)
- char counts (not content)
- daily_tailoring_count, last_tailoring_date

F.1 GET /api/admin/overview

Aspect	Specification
Auth	requireAdmin() — all 3 layers
Purpose	Returns platform-wide aggregate statistics for the overview dashboard.
Data Returned	{ totalUsers: number, totalTailorings: number, tailoringsToday: number, avgAtsScore: number, emailsSentTotal: number, arabicPercentage: number, activeUsersLast7Days: number, topTemplates: {template: string, count: number}[], topStrengths: {strength: string, count: number}[] }
DB Queries	All queries use COUNT(), AVG(), GROUP BY on resumes and profiles tables. No per-user data. All aggregated.
Audit Log	Action: VIEW_OVERVIEW
Cache	Cache response for 60 seconds (Next.js unstable_cache or revalidate: 60) to avoid hammering the DB on repeated refreshes.

F.2 GET /api/admin/users

Aspect	Specification
Auth	requireAdmin()
Purpose	Returns a paginated list of users with usage metadata. No resume content.
Query Params	page (default 1), limit (default 25, max 50), search (email prefix search), sortBy (created_at | tailoring_count), order (asc | desc)
Data Returned (per user)	{ id, email, full_name, language, created_at, daily_tailoring_count, last_tailoring_date, total_tailorings (COUNT from resumes), avg_ats_score (AVG from resumes), email_notifications }
Forbidden Fields	No resume content, no job descriptions, no file URLs.

Implementation	LEFT JOIN profiles with COUNT/AVG subquery from resumes, grouped by user_id. Never expose SUPABASE_SERVICE_ROLE_KEY in response.
Audit Log	Action: VIEW_USERS_LIST, metadata: { page, search_used: boolean }

F.3 GET /api/admin/users/[userId]

Aspect	Specification
Auth	requireAdmin()
Purpose	Returns usage profile for a single user. Still no resume content.
Data Returned	Profile fields (as above) + tailoring_history: array of { id, job_title, company_name, ats_score, language, template_used, email_sent, created_at, original_filename, char_counts: { resume: number, job: number } }. The char_counts are derived from LENGTH(original_text) and LENGTH(job_description) — numbers only, not the text itself.
Forbidden Fields	original_text, job_description, tailored_json, tailored_pdf_url, original_file_url, keywords_matched, keywords_missing.
Implementation	Two queries: one for profile, one for resumes (SELECT only permitted columns). Merge in application layer.
Audit Log	Action: VIEW_USER_DETAIL, targetId: userId

F.4 GET /api/admin/usage

Aspect	Specification
Auth	requireAdmin()
Purpose	Returns time-series usage data for charts. Fully anonymized — no per-user breakdown.
Query Params	period: '7d' | '30d' | '90d' (default '30d')
Data Returned	Array of daily buckets from usage_stats DB view: { day, active_users, total_tailorings, avg_ats_score, emails_sent, arabic_tailorings, english_tailorings }
Implementation	Query the usage_stats DB view filtered by the date range. Service role only.
Audit Log	Action: VIEW_USAGE_STATS, metadata: { period }

F.5 GET /api/admin/activity

Aspect	Specification
Auth	requireAdmin()

Purpose	Returns recent platform activity feed — recent tailorings across all users. Shows what's happening without exposing resume content.
Data Returned	Array (max 50): { id, user_email (masked — show first 3 chars + *** + @domain), job_title, company_name, ats_score, language, template_used, created_at }. Email masking: 'hussain@gmail.com' → 'hus***@gmail.com'.
Forbidden Fields	All resume/job content fields.
Audit Log	Action: VIEW_RECENT_ACTIVITY

F.6 GET /api/admin/audit-log

Aspect	Specification
Auth	requireAdmin()
Purpose	Admin can view their own audit log — what they have accessed and when. Transparency for the admin themselves.
Data Returned	Paginated array of admin_audit_log rows for the authenticated admin's ID only: { id, action, target_id, metadata, ip_address, created_at }
Note	This is the only route that returns from the audit log table. It returns only the calling admin's own records.
Audit Log	This route does NOT log itself (to avoid infinite recursion). All other routes must log.

G. Admin UI — Page Specifications

The Admin Dashboard is a separate section of the app, accessible only when logged in as admin. It uses the same design system (dark theme, DM Sans, Bebas Neue, shadcn/ui components, same CSS tokens) as the main app but with a distinct visual signal: a persistent purple left-border accent on the sidebar and a 'Admin View' badge in the header. This makes it immediately obvious you are in admin mode.

G.1 Admin Routes & Navigation

Route	Page	Description
/admin	Overview	Platform stats, KPI cards, quick-glance charts
/admin/users	User Management	Paginated user list with search and sort
/admin/users/[userId]	User Detail	Single user's usage profile (no resume content)
/admin/usage	Usage Analytics	Time-series charts of tailoring activity

/admin/activity	Recent Activity	Live feed of recent tailorings (masked emails)
/admin/audit	My Audit Log	Admin's own access history

Admin sidebar: separate sidebar component (AdminSidebar.tsx) that replaces the main Sidebar.tsx when in /admin/* routes. Shares the same visual structure but with a purple (--color-admin-accent: #7C3AED) left border on the active nav item instead of blue. Top of sidebar: 'ADMIN' label in Bebas Neue 14px uppercase purple, above the VIGILANTE wordmark. Bottom: 'Exit Admin' button that navigates to /dashboard.

G.2 /admin — Overview Page

Persistent Admin Warning Banner

- A full-width banner pinned at the top of every admin page (inside the content area, below the header):
  - Background: rgba(124, 58, 237, 0.08). Border-left: 3px solid #7C3AED. Text: 'Admin view — all access is logged and audited.' Small ShieldCheck icon (16px, purple). This is non-dismissible.

KPI Cards Row (4 cards)

- Card 1 — Total Users: large number (Bebas Neue 48px, white), label 'Registered Users', Users icon (teal). Shows total count from profiles.
- Card 2 — Total Tailorings: number, label 'Resumes Tailored', Wand2 icon (blue). Total count from resumes.
- Card 3 — Today's Activity: number, label 'Tailorings Today', Activity icon (gold). Count from today's resumes.
- Card 4 — Avg ATS Score: number + '%', label 'Avg ATS Score (All Time)', BarChart2 icon (green). Colored by score: red < 50, gold 50–79, green ≥ 80.
- Cards use --bg-surface, --radius-md, --shadow-card. Each card animates in with a count-up from 0 to the final number (800ms, ease-out) using requestAnimationFrame.

Secondary Stats Row (4 smaller cards)

- Emails Sent Total | Arabic Tailorings % | Most Used Template | Most Used Strength

Tailoring Trend Chart

- A line chart (recharts LineChart) showing tailorings per day for the last 30 days. X-axis: dates formatted 'Apr 3'. Y-axis: count. Teal line, dark grid lines matching --border-subtle. Tooltip shows: date, total tailorings, active users, avg ATS score.
- Period selector: 3-segment control '7D | 30D | 90D'. On change, re-fetches /api/admin/usage.

Language & Template Split (2 small charts)

- Left: A simple donut chart (recharts PieChart) showing EN vs AR split. Teal for EN, gold for AR. Center label: total count.
- Right: A horizontal bar chart showing template usage distribution: Classic | Modern | Arabic RTL.

G.3 /admin/users — User List

- Page title: 'Users' (H1). Subtitle: 'N registered users' — count label.
- Toolbar: Search input (placeholder: 'Search by email...'), Sort dropdown ('Newest | Most Active | Oldest'), Limit select (25 | 50).
- Table columns: Email | Name | Joined | Total Tailorings | Avg ATS | Last Active | Lang | Actions
- Email column: displayed in full (this is the only page where full email is visible, as it is necessary for account management context).
- Actions column: single icon button — ChevronRight — navigates to /admin/users/[userId].
- Pagination: Prev / Next buttons at the bottom. Shows 'Showing 1–25 of N users'.
- Rows are clickable (entire row) — navigates to user detail.
- Empty search state: 'No users matching that email.' with search icon.

G.4 /admin/users/[userId] — User Detail

> **Content Restriction Reminder**
> This page shows usage metadata only. No resume content, no job descriptions, no file URLs. If any team member asks 'why can't the admin read the resumes?' — this is intentional, documented, and non-negotiable.

- Back button: '← Back to Users' (Ghost, small).
- User header card: Avatar (initials, 48px, purple bg in admin mode), full name, email (full), language badge, joined date, 'Admin View' label.
- Stats row: 4 mini KPI cards specific to this user — Total Tailorings, Avg ATS Score, Last Tailoring Date, Email Notifications (on/off badge).
- Tailoring History table: columns — Date | Job Title | Company | ATS Score | Language | Template | Email Sent | Resume Size (chars).
  - ATS Score: colored badge (red/gold/green by threshold).
  - Email Sent: green checkmark or gray dash.
  - Resume Size: displayed as '1,240 chars' — a number, not the content.
  - No 'View Resume' button. No link to the PDF. No resume content of any kind.
- Pagination: 'Load more' ghost button if user has > 25 tailorings.

G.5 /admin/usage — Usage Analytics

- Full-width trend chart (same as overview but larger, full page width, more detail).
- Period selector: 7D | 30D | 90D.
- Below the main chart: 3 side-by-side smaller charts — Active Users per day, Avg ATS Score trend, EN vs AR ratio over time.
- A data table below the charts: each row = one day of data. Columns: Date | Tailorings | Active Users | Avg ATS | Emails Sent | EN Count | AR Count. Sortable. Exportable? NO — no export button anywhere.

G.6 /admin/activity — Recent Activity

- Page title: 'Recent Activity'. Subtitle: 'Last 50 tailoring events across all users.'
- Activity feed: vertical list of event cards. Each card: timestamp (JetBrains Mono, teal, xs) + masked email ('hus***@gmail.com') + 'tailored for [job_title] at [company_name]' + ATS score badge + language badge + template badge.
- No actions on each card — read only.
- Auto-refreshes every 60 seconds with a subtle 'Updated X seconds ago' label (top right, xs, --text-tertiary).
- Filter: Language (All | EN | AR), Template (All | Classic | Modern | Arabic RTL).

G.7 /admin/audit — My Audit Log

- Page title: 'My Audit Log'. Subtitle: 'Every admin action you have taken is recorded here.'
- Explainer callout (teal): 'This log exists to ensure admin access is transparent and accountable. These records cannot be deleted.'
- Table: Date | Action | Target User ID (truncated, not email) | IP Address | Additional Context.
- Read-only. No filter. No search. Sorted newest first. Paginated (25 per page).

H. File Structure Additions

Add the following to the project structure defined in Section 3 of the main build prompt:

```
src/
├── app/
│   └── admin/                          ← NEW
│       ├── layout.tsx                  ← Admin shell (AdminSidebar + audit
banner)
│       ├── page.tsx                    ← /admin overview
│       ├── users/
│       │   ├── page.tsx                ← /admin/users list
│       │   └── [userId]/page.tsx       ← /admin/users/[id] detail
│       ├── usage/page.tsx              ← /admin/usage analytics
│       ├── activity/page.tsx           ← /admin/activity feed
│       └── audit/page.tsx             ← /admin/audit log
│
│   └── api/admin/                      ← NEW
│       ├── overview/route.ts
│       ├── users/route.ts
│       ├── users/[userId]/route.ts
│       ├── usage/route.ts
│       ├── activity/route.ts
│       └── audit-log/route.ts
```

```
│
├── components/app/
│   ├── AdminSidebar.tsx                ← NEW
│   ├── AdminWarningBanner.tsx          ← NEW (persistent, non-dismissible)
│   ├── KpiCard.tsx                     ← NEW (reusable stat card)
│   ├── TailoringTrendChart.tsx         ← NEW (recharts line chart)
│   ├── LanguageSplitChart.tsx          ← NEW (recharts donut)
│   ├── TemplateSplitChart.tsx          ← NEW (recharts bar)
│   └── ActivityFeed.tsx                ← NEW (recent activity list)
│
├── lib/admin/                          ← NEW
│   ├── require-admin.ts                ← 3-layer auth check
│   ├── audit-logger.ts                 ← logAdminAction()
│   └── data-formatters.ts             ← maskEmail(), formatCharCount(), etc.
│
├── hooks/
│   └── useAdminData.ts                 ← NEW (React Query hooks for admin
API)
│
└── types/
    └── admin.ts                        ← NEW (AdminUser, UsageStat,
ActivityEvent types)

supabase/migrations/
└── 004_admin_role_and_audit.sql        ← NEW

tests/
├── unit/
│   ├── admin-require-admin.test.ts     ← NEW
│   ├── admin-audit-logger.test.ts      ← NEW
│   └── admin-data-formatters.test.ts   ← NEW
└── integration/
    ├── api-admin-overview.test.ts      ← NEW
    ├── api-admin-users.test.ts         ← NEW
    └── api-admin-access-control.test.ts ← NEW (most important)
```

I. Testing Requirements — Admin Dashboard

I.1 Unit Tests

Test File	What It Tests
admin-require-admin.test.ts	(1) No session → returns 401 response object. (2) Valid session, is_admin=false → returns 403. (3) Valid session, is_admin=true, email mismatch → returns 403. (4) Valid session, is_admin=true, email matches → returns { adminId, adminEmail, error: null }. All 4 paths explicitly tested.

admin-audit-logger.test.ts	logAdminAction() inserts correct record. All fields present: admin_id, action, ip_address, created_at. With targetId: target_id is set. Without: target_id is null. Supabase client is mocked.
admin-data-formatters.test.ts	maskEmail('hussain@gmail.com') → 'hus***@gmail.com'. maskEmail('ab@x.com') → 'ab***@x.com' (handles short prefixes). formatCharCount(1240) → '1,240 chars'. ATS score color: 79 → gold, 80 → green, 49 → red.

I.2 Integration Tests — Access Control (Most Critical)

These tests are the most important in the entire test suite for the admin feature.
They verify that non-admin users CANNOT access admin data under any circumstances.

Test File	What It Tests
api-admin-access-control.test.ts	(1) Unauthenticated request to any /api/admin/* → 401. (2) Authenticated regular user to any /api/admin/* → 403. (3) User with is_admin=true but wrong email → 403. (4) Admin user → 200 on all routes. (5) Admin response for /api/admin/users never contains original_text, job_description, tailored_json, tailored_pdf_url. (6) Admin response for /api/admin/users/[id] never contains those fields. (7) Email masking verified on /api/admin/activity response. Test all 6 admin routes for cases 1–4.
api-admin-overview.test.ts	Returns all expected aggregate fields. Numbers are non-negative. avg_ats_score is 0–100. Audit log entry created after successful call.
api-admin-users.test.ts	Pagination works (page 2 returns next 25). Search filters by email prefix. Sort by tailoring_count works. Response never contains forbidden fields (explicit assertion for each forbidden field name).

I.3 E2E Tests

- Add tests/e2e/admin.spec.ts with these flows:
  - Non-admin user navigating to /admin → redirected to /dashboard (not a 404, a redirect).
  - Admin user logs in → navigates to /admin → Overview page loads with KPI cards.
  - Admin user navigates to /admin/users → user list renders with masked emails in Activity, full emails in Users list.
  - Admin user clicks a user → detail page shows tailoring history without any resume content links or buttons.
  - Admin user navigates to /admin/audit → their own audit log entries appear (generated by previous steps in the test).
  - Admin user clicks 'Exit Admin' → navigates to /dashboard → admin sidebar is gone.

J. Phase Integration — Where Each Part Gets Built

Phase	Admin Work Added
Phase 0 — Foundation	Apply Migration 004. Add ADMIN_EMAIL and ADMIN_HMAC_SECRET to .env.local. Output ACTION REQUIRED for admin account setup. Create /lib/admin/ folder skeleton. Create /types/admin.ts with all admin types.
Phase 1 — Infrastructure	Add admin route protection to middleware.ts. Build AdminSidebar.tsx and AdminWarningBanner.tsx. Build admin layout (app/admin/layout.tsx) with the persistent banner. Set up admin navigation items.
Phase 2 — API Pipeline	Build all 6 admin API routes (/api/admin/*). Build require-admin.ts, audit-logger.ts, data-formatters.ts. Write unit + integration tests for admin.
Phase 4 — UI Integration	Build all 6 admin pages. Build KpiCard, TailoringTrendChart, LanguageSplitChart, TemplateSplitChart, ActivityFeed components. Build useAdminData.ts React Query hooks. Wire all pages to their API routes.
Phase 5 — Security & QA	Admin-specific security audit: verify all 3 auth layers, verify no forbidden fields in responses, verify audit log is created on every admin call, run api-admin-access-control.test.ts, run admin E2E tests, verify admin routes return 403 for non-admin users.
Phase 6 — Deployment	Verify ADMIN_EMAIL and ADMIN_HMAC_SECRET are set in Vercel production environment. Run admin E2E tests against production URL. Confirm audit log is writing to production DB. Update /docs/final.md with admin section.

K. Admin Feature Completion Checklist

The agent must verify every item before declaring the admin dashboard feature complete:

Security

- requireAdmin() is called as the first line of every admin API route handler.
- logAdminAction() is called in every admin API route before returning a response.
- No admin API route returns: original_text, job_description, tailored_json, tailored_pdf_url, original_file_url.
- api-admin-access-control.test.ts — all tests pass.
- Non-admin user visiting /admin/* is redirected, not shown a 403 error page.
- ADMIN_EMAIL is a server-only env var (not prefixed NEXT_PUBLIC_).
- is_admin column can only be set via direct SQL — no API endpoint reads or writes it directly.

Privacy

- Email addresses on /admin/activity are masked ('hus***@gmail.com' format).
- No resume content, no job description text, no file URLs visible anywhere in the admin UI.
- No export/download button exists anywhere in the admin UI.
- AdminWarningBanner.tsx is present and non-dismissible on every admin page.

Functionality

- All 6 admin pages render correctly with real data.
- KPI cards show accurate aggregated numbers.
- TailoringTrendChart renders for 7D, 30D, and 90D periods.
- User list search, sort, and pagination work correctly.
- User detail page shows tailoring history with char counts but no content.
- Audit log page shows the admin's own access history.
- 'Exit Admin' button navigates to /dashboard correctly.

Tests

- All 3 unit test files pass.
- All 3 integration test files pass — especially api-admin-access-control.test.ts.
- admin.spec.ts E2E tests pass on Chromium and Mobile Safari.

Final Note on the Admin Dashboard

The admin dashboard is built to serve one person: you, the builder. Its purpose is operational awareness — knowing the system is healthy, being able to diagnose issues, and understanding how people are using the product.

It is NOT a surveillance tool. The data restrictions in this document exist to protect your users' trust. A resume contains some of the most personal career information a person has. That data was given to the app to be tailored, not to be read by an admin.

Build it with that principle at the center, and it will be the right admin dashboard.